

Parsing, Code Generation & Optimization

Compilers

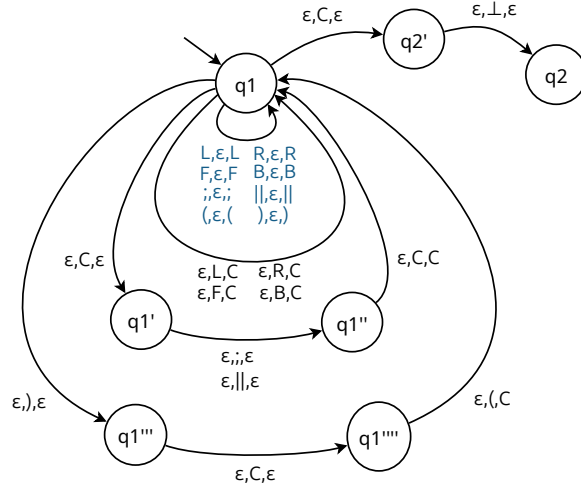
1. (a) This grammar is ambiguous because:

- No associativity is enforced for $;$: $C_1; C_2; C_3$ can for example be derived using two different parse trees corresponding to $C_1; (C_2; C_3)$ and $(C_1; C_2); C_3$.
- No associativity is enforced for $\parallel$: $C_1 \parallel C_2 \parallel C_3$ can for example be derived using two different parse trees corresponding to $C_1 \parallel (C_2 \parallel C_3)$ and $(C_1 \parallel C_2) \parallel C_3$.
- No precedence is enforced between $\parallel$ and $;$: $C_1 \parallel C_2; C_3$ can for example be derived using two different parse trees corresponding to $C_1 \parallel (C_2; C_3)$ and $(C_1; C_2) \parallel C_3$.

Those ambiguities can be fixed by fixing the precedence and associativity of the operators. For example, requiring that the operators are right associative, and that $\parallel$ has precedence over $;$:

$$\begin{aligned} C &::= D; C \mid D \\ D &::= E \parallel D \mid (E) \\ E &::= L \mid R \mid F \mid B \mid (E) \end{aligned}$$

(b) Here is a PDA that captures a bottom-up parser for this language, where the initial stack symbol is $\perp$, the blue transition are shifts, and the black transitions reduce:



(c) The following computation shows that $F; L \parallel R$ is accepted by this PDA:

- $(q_1, F; L \parallel R, \perp)$
- $\rightarrow (q_1, ; L \parallel R, F \perp)$ – shift
- $\rightarrow (q_1, ; L \parallel R, C \perp)$ – reduce
- $\rightarrow (q_1, L \parallel R, ; C \perp)$ – shift
- $\rightarrow (q_1, \parallel R, L; C \perp)$ – shift
- $\rightarrow (q_1, \parallel R, C; C \perp)$ – reduce
- $\rightarrow (q_1, R, \parallel C; C \perp)$ – **shift**

- viii. $\rightarrow (q_1, \epsilon, R \| C; C \perp) - \text{shift}$
- ix. $\rightarrow (q_1, \epsilon, C \| C; C \perp) - \text{reduce}$
- x. $\rightarrow (q'_1, \epsilon, \| C; C \perp)$
- xi. $\rightarrow (q''_1, \epsilon, C; C \perp)$
- xii. $\rightarrow (q_1, \epsilon, C; C \perp) - \text{reduce}$
- xiii. $\rightarrow (q'_1, \epsilon, ; C \perp)$
- xiv. $\rightarrow (q''_1, \epsilon, C \perp)$
- xv. $\rightarrow (q_1, \epsilon, C \perp) - \text{reduce}$
- xvi. $\rightarrow (q'_2, \epsilon, \perp)$
- xvii. $\rightarrow (q_2, \epsilon, \epsilon)$

- (d) A shift/reduce ambiguity arises at step 7, where instead of shifting $\|$, $C; C$ could have been reduced to C . Shifting $\|$ leads to the parsing of $F; (L \| R)$ while reducing $C; C$ leads to the parsing of $(F; L) \| R$.

2. (a) It can for example be translated as follows:

statement	instructions	register descriptors			address descriptors				cost
		<i>R1</i>	<i>R2</i>	<i>R3</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	
$a = b$	LD <i>R1</i> , <i>b</i>	<i>b, a</i>			<i>R1</i>	<i>b, R1</i>	<i>c</i>	<i>d</i>	2
$b = 2 * a$	MUL <i>R2</i> , #2, <i>R1</i>	<i>a</i>	<i>b</i>		<i>R1</i>	<i>R2</i>	<i>c</i>	<i>d</i>	2
$c = a$		<i>a, c</i>	<i>b</i>		<i>R1</i>	<i>R2</i>	<i>R1</i>	<i>d</i>	0
$d = c + c$	ADD <i>R3</i> , <i>R1</i> , <i>R1</i>	<i>a, c</i>	<i>b</i>	<i>d</i>	<i>R1</i>	<i>R2</i>	<i>R1</i>	<i>R3</i>	1
	ST <i>a</i> , <i>R1</i>								
	ST <i>b</i> , <i>R2</i>								
end	ST <i>c</i> , <i>R1</i>	<i>a, c</i>	<i>b</i>	<i>d</i>	<i>a, R1</i>	<i>b, R2</i>	<i>c, R1</i>	<i>d, R3</i>	4 * 2
	ST <i>d</i> , <i>R3</i>								

- (b) Here is one way we can optimize the code:

- We start with:

$a = b$
 $b = 2 * a$
 $c = a$
 $d = c + c$

- We can do a copy propagation of c , resulting in:

$a = b$
 $b = 2 * a$
 $c = a$
 $d = a + a$

- We can then transform $2 * a$ into $a + a$:

$a = b$
 $b = a + a$
 $c = a$
 $d = a + a$

- We can then use local common subexpression as $a + a$ is done twice:

$$\begin{aligned}a &= b \\ b &= a + a \\ c &= a \\ d &= b\end{aligned}$$

- We can then do a copy propagation of a :

$$\begin{aligned}a &= b \\ b &= b + b \\ c &= a \\ d &= b\end{aligned}$$

- (c) As mentioned in the table above, the cost of the original code is 13. The cost of the optimized code would be 11 as there wouldn't be a #2 anymore as the multiplication would be replaced by an addition, and the addition would go away. Therefore, the optimized version is less costly.